ROKOCO

PLA.NET

ALM.IT Liability-Modell

Version 1.8

Grünwald 28.11.2025

© 2025 ROKOCO GmbH

Versionshistorie

Version 1.0	Erstversion	Sebastian Hantsch	10.04.2018
Version 1.1	Pivotwürfel: Standardkategorien/-ausprägungen ergänzt	Sebastian Hantsch	18.06.2018
Version 1.2	PLA.NET 5.2 Anpassungen	Sebastian Hantsch	20.04.2020
Version 1.3	PLA.NET 5.3 Anpassungen	Sebastian Hantsch	06.08.2020
Version 1.4	PLA.NET 6.2 Anpassungen	Sebastian Hantsch	01.06.2021
Version 1.5	Muster-Transportfunktion	Sebastian Hantsch	02.03.2022
Version 1.6	PLA.NET 8.3 Anpassungen	Sebastian Hantsch	22.09.2023
Version 1.7	PLA.NET 10.3 Anpassungen	Sebastian Hantsch	01.08.2025
Version 1.8	BSM 4.4 Anpassungen VT Neu Initialwerte	Sebastian Hantsch	16.09.2025

Inhaltsverzeichnis

1. Einführung ... 1
2. Referenz ... 2
2.1 Access-Import ... 2
2.2 Any ... 4
2.3 Dmat ... 7
2.4 Interaktion ... 9
2.5 Excelimport ... 13
2.6 Programmablaufsteuerung ... 14
2.7 CSV-Logs ... 14
2.8 Ltab ... 15
2.9 Mathematische Funktionen ... 17
2.10 Memory Hub ... 20
2.11 Pivotwürfel ... 22
2.12 Rset ... 31
2.13 Strings ... 35
2.14 Stringlist ... 36
2.15 Svec ... 36
3. Transportfunktion ... 37
3.1 Standard-Zuordnung zum Leistungsspektrum ... 37
3.2 Mustertransport-Funktion ... 39

1. Einführung

Im Zuge der Zusammenführung der ALM-Systeme PLA.NET und ALM.IT wurde die komplette Liabilitiy-Modellierung von ALM.IT in PLA.NET integriert. Dieses Dokument dient zur Dokumentation dieses Moduls.

2. Referenz

2.1 Access-Import

Die folgenden Methoden dienen zum direkten Datenimport aus Microsoft Access-Datenbanken. Hierbei wird das aktuelle Access-Datenbank-Format accdb (Standard seit Microsoft Access 2007) unterstützt. Damit diese Importschnittstelle verwendet werden kann, muss auf dem Client-PC eine der folgenden Bedingungen erfüllt sein:

- Installation der 64bit-Version von Microsoft Access Database Engine 2010 Redistributable (oder neuere Version)
- Installation der 64bit-Version von Microsoft Access 2007 oder neuer

Diese Methoden können nur in Access Import Modulen verwendet werden.

2.1.1 Methoden

```
FUNCTION AccessGetTables(): SVEC
```

Gibt eine Liste der verfügbaren Tabellen in der Access-Datenbank zurück.

```
PROCEDURE AccessOpenTable(TableName: STRING)
```

Öffnet die angegebene Tabelle in der aktuellen Access-Datenbank.

```
FUNCTION AccessGetColumns(): SVEC
```

Gibt eine Liste der Spaltennamen in der geöffneten Tabelle der aktuellen Access-Datenbank zurück.

```
FUNCTION AccessGetColumnType(Index: INTEGER): STRING
```

Gibt den Datentyp der angegebenen Spalte in der geöffneten Tabelle der aktuellen Access-Datenbank zurück. Mögliche Rückgabewerte sind hierbei:

- Integer
- Real
- Boolean
- String

```
FUNCTION AccessGetColumnSize(Index: INTEGER): INTEGER
```

Gibt die maximale Länge der angegebenen Spalte in der geöffneten Tabelle der aktuellen Access-Datenbank zurück. Dies ist im Allgemeinen nur für String-Spalten sinnvoll.

```
PROCEDURE AccessRewind()
```

Setzt die aktuelle Position in der geöffneten Tabelle auf den Anfang.

```
FUNCTION AccessRead (): BOOLEAN
```

Lädt die nächste Zeile aus der geöffneten Tabelle. Sofern es noch eine Zeile gab, gibt die Funktion Wahr zurück, ansonsten Falsch.

```
FUNCTION AccessGetInteger(Index: INTEGER): INTEGER
```

Liest den Wert einer Integer-Spalte aus der geöffneten Tabelle aus.

```
FUNCTION AccessGetBoolean(Index: INTEGER): BOOLEAN
```

Liest den Wert einer Boolean-Spalte aus der geöffneten Tabelle aus.

```
FUNCTION AccessGetReal(Index: INTEGER): REAL
```

Liest den Wert einer Real-Spalte aus der geöffneten Tabelle aus.

```
FUNCTION AccessGetString(Index: INTEGER): STRING
```

Liest den Wert einer String-Spalte aus der geöffneten Tabelle aus.

2.1.2 Beispiel

Das folgende Beispiel importiert eine Access-Tabelle namens „Bestand“ in einen neuen Recordset.

```
Var
  dt: string;
  rs: rset;
  cols, coltypes: svec;
  i: integer;
begin
   rinit(rs, ‘Bestand’);

   AccessOpenTable(‘Bestand’);
   cols := AccessGetColumns();
   svecresize(coltypes, svecsize(cols));

   for i := 0 to svecsize(cols) - 1 do
   begin
     dt := AccessGetColumnType(i);
     coltypes[i] := dt;
     if dt = ‘Integer’ then
       raddcolinteger(rs, cols[i])
     else if dt = ‘Real’ then
       raddcolreal(rs, cols[i])
     else if dt = ‘Boolean’ then
       raddcolboolean(rs, cols[i])
     else
       raddcolstringsize(rs, cols[i], AccessGetColumnSize(i));
   end;
```

```
  AccessRewind();
  while AccessRead do
  begin
    raddrow(rs);

    for i := 0 to svecsize(cols) - 1 do
    begin
      dt := coltypes[i];
      if dt = 'Integer' then
        rsetinteger(rs, i, AccessGetInteger(i))
      else if dt = 'Real' then
        rsetreal(rs, i, AccessGetReal(i))
      else if dt = 'Boolean' then
        rsetboolean(rs, i, AccessGetBoolean(i))
      else
        rsetstring(rs, i, AccessGetString(i));
    end;
  end;

  rstore(rs, 'Access-Bestand');
end;
```

2.2 Any

2.2.1 Methoden

```
FUNCTION AnyGetName(a: ANY): STRING
```

Gibt den Namen des Any-Werts a als String zurück.

```
FUNCTION AnySetName(VAR a: ANY; sAnyName: STRING): BOOLEAN
```

Ändert den Namen des Any-Werts a auf den String sAnyName. Die Funktion gibt Wahr zurück, falls der Name erfolgreich geändert werden konnte, ansonsten Falsch.

```
FUNCTION AnyGetValue(a: ANY): STRING
```

Gibt den Wert des Any-Werts a als String zurück.

```
FUNCTION AnySetValue(VAR a: ANY; value: STRING): BOOLEAN
```

Setzt den Wert des Any-Werts a auf den String value. Die Funktion gibt Wahr zurück, falls der Wert erfolgreich geändert werden konnte, ansonsten Falsch.

```
FUNCTION AnySetRealValue(VAR a: ANY; value: REAL): BOOLEAN
```

Setzt den Wert des Any-Werts a auf den Real-Wert value. Die Funktion gibt Wahr zurück, falls der Wert erfolgreich geändert werden konnte, ansonsten Falsch.

```
FUNCTION AnySetIntegerValue(VAR a: ANY; value: INTEGER): BOOLEAN
```

Setzt den Wert des Any-Werts a auf den Integer-Wert value. Die Funktion gibt Wahr zurück, falls der Wert erfolgreich geändert werden konnte, ansonsten Falsch.

```
FUNCTION AnySetBooleanValue(VAR a: ANY; value: BOOLEAN): BOOLEAN
```

Setzt den Wert des Any-Werts a auf den Boolean-Wert value. Die Funktion gibt Wahr zurück, falls der Wert erfolgreich geändert werden konnte, ansonsten Falsch.

```
FUNCTION AnySetDateValue(VAR a: ANY; value: DATE): BOOLEAN
```

Setzt den Wert des Any-Werts a auf den Date-Wert value. Die Funktion gibt Wahr zurück, falls der Wert erfolgreich geändert werden konnte, ansonsten Falsch.

```
FUNCTION AnyIsString(a: ANY): BOOLEAN
```

Ermittelt, ob der übergebene Any-Wert einen String enthält.

```
FUNCTION AnyIsEmpty(a: ANY): BOOLEAN
```

Ermittelt, ob der übergebene Any-Wert leer ist.

```
FUNCTION AnyToString(a: ANY): STRING
```

Wandelt den Wert von a in einen String um und gibt diesen zurück.

```
FUNCTION AnyToReal(a: ANY): REAL
```

Wandelt den Wert von a in Real um und gibt dieses zurück.

```
FUNCTION AnyToInteger(a: ANY): INTEGER
```

Wandelt den Wert von a in Integer um und gibt dieses zurück.

```
FUNCTION AnyToBoolean(a: ANY): BOOLEAN
```

Wandelt den Wert von a in Boolean um und gibt dieses zurück.

```
FUNCTION AnyToDate(a: ANY): DATE
```

Wandelt den Wert von a in Date um und gibt dieses zurück.

```
FUNCTION anyislist(a: ANY): BOOLEAN
```

Ermittelt, ob der übergebene Any-Wert eine Liste enthält.

```
PROCEDURE AnyToAnylist(ParAny: ANY; VAR ParAnyList: ANYLIST)
```

Speichert den Listeninhalt des Any-Werts ParAny in der übergebenen Anylist.

```
FUNCTION AnyListGetAny(AnyList: ANYLIST; sAnyName: STRING): ANY
```

Sucht in AnyList nach einem Any-Element mit dem Namen sAnyName und gibt das erste gefundene Any mit diesem Namen zurück.

```
FUNCTION AnyListGetIter(AnyList: ANYLIST): ANYLISTITER
```

Gibt einen Iterator für die Liste AnyList zurück.

```
FUNCTION AnyListGetNameList(AnyList: ANYLIST): STRINGLIST
```

Gibt eine Liste der Namen aller Elemente der übergebenen Anylist zurück.

```
FUNCTION AnyListIsIterValid(AnyList: ANYLIST; AnyLIter: ANYLISTITER):
BOOLEAN
```

Überprüft ob der Iterator AnyLIter gültig ist.

```
FUNCTION AnyListToAny(AnyList: ANYLIST): ANY
```

Erzeugt ein neues Any, setzt dessen Inhalt auf die Liste AnyList und gibt es dann zurück.

```
PROCEDURE AnyListIterstep(VAR AnyLIter: ANYLISTITER)
```

Setzt den Iterator AnyLIter auf das nächste Element.

```
FUNCTION AnyListIterGetItem(AnyLIter: ANYLISTITER): ANY
```

Gibt das aktuelle Element an der Position des Iterators zurück.

```
PROCEDURE AnyListAppend(VAR AnyList: ANYLIST; ParAny: ANY)
```

Fügt der Liste AnyList am Ende das Element ParAny hinzu.

```
PROCEDURE AnylistClear(VAR AnyList: ANYLIST)
```

Entfernt alle Elemente aus der übergebenen AnyList.

```
FUNCTION NewAnySetRealValue(value: REAL): ANY
```

Erzeugt ein neues Any vom Typ Real und Wert value und gibt dieses Any zurück.

2.3 Dmat

2.3.1 Methoden

```
PROCEDURE dmatreshape(mDMat: DMAT; iRow, iCol: INTEGER)
```

Redimensioniert die übergebenene dmat mit der übergebenen Zeilen- und Spaltenanzahl.

```
FUNCTION dmatrows(mDMat: DMAT): INTEGER
```

Gibt die Zeilenanzahl der übergebenen dmat zurück.

```
FUNCTION dmatcols(mDMat: DMAT): INTEGER
```

Gibt die Spaltenanzahl der übergebenen dmat zurück.

```
PROCEDURE dmatredim(VAR mDMat: DMAT; iRow, iCol: INTEGER; fillWithDefault:
BOOLEAN)
```

Redimensioniert die übergebene dmat mit der übergebenen Zeilen- und Spaltenanzahl. Falls gewünscht, werden alle Inhalte der dmat mit dem Default-Wert überschrieben, andernfalls nur die neuen Zeilen und Spalten.

```
PROCEDURE dmatselectrow(mDMat: DMAT; iRow: INTEGER; VAR mDMatRow: DMAT)
```

Kopiert die angebene Zeile der übergebenen dmat in eine neue dmat.

```
PROCEDURE dmatselectcol(mDMat: DMAT; iCol: INTEGER; VAR mDMatRow: DMAT)
```

Kopiert die angebene Spalte der übergebenen dmat in eine neue dmat.

```
PROCEDURE dmatfill(mDMat: DMAT; dFillElem: REAL)
```

Füllt alle Elemente der übergebenen dmat mit dem Wert dFillElem.

```
FUNCTION dmatisvalid(mDMat: DMAT): BOOLEAN
```

Prüft, ob die übergebenene dmat gültig ist.

```
FUNCTION dmatgetdefault(mDMat: DMAT): REAL
```

Gibt den Default-Wert der übergebenen dmat zurück.

```
PROCEDURE dmatsetdefault(mDMat: DMAT; dDefElem: REAL)
```

Setzt den Default-Wert von mDMat auf dDefElem.

```
FUNCTION dvecisvalid(mDVec: DVEC): BOOLEAN
```

Überprüft ob der übergebene dvec gültig ist.

```
FUNCTION dvecgetdefault(mDVec: DVEC): REAL
```

Gibt den Default-Wert des übergebenen dvec zurück.

```
PROCEDURE dvecsetdefault(mDVec: DVEC; dDefElem: REAL)
```

Setzt den Default-Wert des übergebenen dvec auf dDefElem.

```
FUNCTION dvecsize(mDVec: DVEC): INTEGER
```

Gibt die Größe des übergebenen dvec zurück.

```
PROCEDURE dvecresize(VAR mDVec: DVEC; iRows: INTEGER)
```

Setzt die Größe des übergebenen dvec auf iRows.

```
PROCEDURE dvecfill(mDVec: DVEC; dFillElem: REAL)
```

Füllt den übergebenen dvec mit dem übergebenen Wert.

```
FUNCTION dvecmaxvalue(mDVec: DVEC): REAL
```

Gibt den größten Wert des übergebenen dvecs zurück.

```
FUNCTION dvecsum(mDVec: DVEC): REAL
```

Gibt die Summe aller Werte des übergebenen dvecs zurück.

```
FUNCTION dvecsumofsquares(mDVec: DVEC): REAL
```

Gibt die Summe der Quadratwurzeln aller Werte des übergebenen dvecs zurück.

```
FUNCTION Kurtosis(mDVec: DVEC): REAL
```

Gibt die Wölbung aller Werte des übergebenen dvecs zurück.

```
FUNCTION Skew(mDVec: DVEC): REAL
```

Gibt die Schiefe aller Werte des übergebenen dvecs zurück.

```
FUNCTION dvecnorm(mDVec: DVEC): REAL
```

Gibt die Norm aller Werte des übergebenen dvecs zurück.

```
FUNCTION dvectotalvariance(mDVec: DVEC): REAL
```

Gibt die Gesamtabweichung aller Werte des übergebenen dvecs zurück.

```
FUNCTION dvecpopnstddev(mDVec: DVEC): REAL
```

Gibt die Standardabweichung der Grundgesamtheit des übergebenen dvecs zurück.

```
FUNCTION dvecpopnvariance(mDVec: DVEC): REAL
```

Gibt die Varianz der Grundgesamtheit des übergebenen dvecs zurück.

```
FUNCTION dvecstddev(mDVec: DVEC): REAL
```

Gibt die Standardabweichung der Stichprobe des übergebenen dvecs zurück.

```
FUNCTION dvecvariance(mDVec: DVEC): REAL
```

Gibt die Varianz der Stichprobe des übergebenen dvecs zurück.

```
FUNCTION dvecmean(mDVec: DVEC): REAL
```

Gibt den Mittelwert aller Werte des übergebenen dvecs zurück.

```
PROCEDURE dvecmeanandstddev(mDVec: DVEC; VAR dMean, dStdDev: REAL)
```

Ermittelt Mittelwert und Standardabweichung der Stichprobe des übergebenen dvecs.

```
PROCEDURE dvecmomentskewkurtosis(mDVec: DVEC; VAR dMean, dVariance, dSkew,
dKurtosis: REAL)
```

Ermittelt Mittelwert, Varianz der Stichprobe, Schiefe und Wölbung des übergebenen dvecs.

2.4 Interaktion

2.4.1 Methoden

```
FUNCTION iscompiler(): BOOLEAN
```

Gibt an, ob die Benutzerprogrammierung kompiliert wurde.

```
FUNCTION isprojection(): BOOLEAN
```

Gibt an, ob die Benutzerprogrammierung in einer Projektion ausgeführt wird.

```
FUNCTION ismmpmode(): BOOLEAN
```

Gibt an, ob die Benutzerprogrammierung im MMP-Modus ausgeführt wird.

```
FUNCTION riskfactorgetvalue(sName: STRING; iPath, iYear: INTEGER): REAL
```

Gibt den Wert des angebenen Risikofaktors im angegebenen Kapitalmarktszenario und Jahr zurück.

```
FUNCTION getcurdate(): DATE
```

Gibt das aktuelle Datum der Projektion zurück.

```
FUNCTION getcuryear(): INTEGER
```

Gibt das aktuelle Jahr der Projektion zurück.

```
FUNCTION getcurpath(): INTEGER
```

Gibt die aktuelle Pfadnummer der Projektion zurück.

```
FUNCTION getnumyears(): INTEGER
```

Gibt die Projektionsdauer der Projektion zurück.

```
FUNCTION getmodificationstartyear(): INTEGER
```

Gibt das Modifikationsstartjahr der Projektion zurück.

```
FUNCTION tsgetnum(): INTEGER
```

Gibt die Anzahl der Timesteps der Projektion zurück.

```
FUNCTION getinputdate(): DATE
```

Gibt das Referenzdatum zurück.

```
FUNCTION getfirstprojectiondate(): DATE
```

Gibt das Startdatum der Projektion zurück.

```
PROCEDURE incrcuryear()
```

Erhöht das aktuelle Projektionsjahr um eins.

```
PROCEDURE setcuryear(iYear: INTEGER)
```

Setzt das aktuelle Projektionsjahr auf den übergebenen Wert.

```
FUNCTION getsolvencypart(): INTEGER
```

Gibt den aktuell berechneten Solvency-II-Stress zurück (Nummerierung analog zur Standard-BSM-Nummerierung).

```
FUNCTION getbatchpartindex(): INTEGER
```

Gibt den einsbasierten Index der aktuellen Batchteilberechnung zurück.

```
FUNCTION getbatchpartname(): STRING
```

Gibt den Namen der aktuellen Batchteilberechnung zurück.

```
FUNCTION hasmodule(sModuleName: STRING): BOOLEAN
```

Ermittelt, ob ein Modul mit dem Namen sModuleName existiert.

```
FUNCTION modulegetvaluebyheader(sName, sColumn: STRING; iRow: Integer): ANY
```

Gibt den Wert des angegebenen Moduls in der angebenen Zeile und Spalte zurück.

```
FUNCTION modulegetvalue(sName: STRING; iColumn, iRow: Integer): ANY
```

Gibt den Wert des angegebenen Moduls in der angebenen Zeile und Spalte zurück.

```
FUNCTION getvectattr(sVectorName, sAttrName: STRING; iYear: Integer): ANY
```

Gibt aus dem angegebenen Vektor den Wert des angegebenen Attributs im angegebenen Jahr zurück.

```
FUNCTION setvectattr(aValue: ANY; sVectorName, sAttrName: STRING; iYear:
Integer): BOOLEAN
```

Setzt im angegebenen Vektor den Wert des angegebenen Attributs im angegebenen Jahr.

```
FUNCTION getcnstrattr(sCnstrName, sAttrName: STRING; iYear: Integer): ANY
```

Gibt aus dem angegebenen Regelvorgaben den Wert des angegebenen Attributs im angegebenen Jahr zurück.

```
FUNCTION vecthasattr(sVectorName, sAttrName: STRING): BOOLEAN
```

Überprüft, ob der angegebene Vektor das angegebene Attribut enthält.

```
FUNCTION getassetclassattribute(sAssetClassName, sAttrName: STRING; nIndex,
nIndex2: Integer): STRING
```

Gibt aus der angegebenen Assetklasse für das angegebene Attribut den gewünschten Wert zurück.

```
FUNCTION dategetyearday(d: DATE): INTEGER
```

Gibt Tag des Jahres des angegebenen Datums zurück.

```
FUNCTION dategetweekday(d: DATE): INTEGER
```

Gibt den Wochentag des angegebenen Datums zurück.

```
FUNCTION dategetday(d: DATE): INTEGER
```

Gibt den Tag des Monats des angegebenen Datums zurück.

```
FUNCTION dategetmonth(d: DATE): INTEGER
```

Gibt den Monat des angegebenen Datums zurück.

```
FUNCTION dategetyear(d: DATE): INTEGER
```

Gibt das Jahr des angegebenen Datums zurück.

```
FUNCTION mkdate(d, m, year: Integer): DATE
```

Erzeugt ein neues Datum aus dem angegebenen Tag, Monat und Jahr.

```
FUNCTION datetoday(): DATE
```

Gibt das aktuelle Rechnerdatum zurück.

```
FUNCTION datediff(from, ende: DATE): REAL
```

Gibt die Differenz in ganzen Tagen und Bruchteilen von Tagen zwischen den zwei angegebenen Zeitpunkten zurück.

```
FUNCTION dateadddays(jd: DATE; days: INTEGER): DATE
```

Gibt das übergegebene Datum erhöht um die angegebene Anzahl an Tagen zurück.

```
FUNCTION datesubdays(jd: DATE; days: INTEGER): DATE
```

Gibt das übergegebene Datum verringert um die angegebene Anzahl an Tagen zurück.

```
FUNCTION dateisvalid(d: DATE): BOOLEAN
```

Prüft, ob das angegebene Datum ein gültiges Datum ist.

```
FUNCTION gettickcount(d: DATE): LONG
```

Gibt die Anzahl an Ticks zurück, die die aktuelle Uhrzeit und das aktuelle Datum darstellen.

2.5 Excelimport

Die folgenden Methoden dienen zum direkten Datenimport aus Microsoft Excel-Arbeitsmappen. Hierbei werden die gebräuchlichen Excel-Dateiformate xls, xlsx und xlsm unterstützt.

Diese Methoden können nur in Excel Import Modulen verwendet werden.

2.5.1 Methoden

```
FUNCTION ExcelGetWorksheets(): SVEC
```

Gibt eine Liste der in der zu importierenden Excel-Arbeitsmappe vorhandenen Arbeitsblätter zurück.

```
PROCEDURE ExcelActivateWorksheet(Name: STRING)
```

Aktiviert in der zu importierenden Excel-Arbeitsmappe das angegebene Arbeitsblatt.

```
FUNCTION ExcelGetCell(Row, Column: INTEGER): ANY
```

Gibt den Inhalt der angegebenen Arbeitsblatt-Zelle des aktiven Arbeitsblatts zurück.

2.5.2 Beispiel

Das folgende Beispiel überträgt Daten aus einem Excel-Arbeitsblatt in eine neue Liability Table.

```
Var
  lt: ltab;
  b: boolean;
  a: any;
  row, col: integer;
begin
   ltabinit(lt, 'Excelimporttest', 10, 10);

   ExcelActivateWorksheet('Tabelle1');

   for col := 1 to 2 do
   begin
      AnySetName(a, AnyToString(ExcelGetCell(1, col)));
      b := ltabsetheadnum(lt, a, col - 1);

      for row := 1 to 10 do
      begin
        b := ltabsetrc(lt, ExcelGetCell(row + 1, col), row - 1, col - 1);
      end;
   end;

   ltabstore(lt, 'Excelimporttest');
end;
```

2.6 Programmablaufsteuerung

2.6.1 Methoden

```
PROCEDURE execute(name: STRING)
```

Startet eine benutzerdefinierte Funktion. Die Funktion wird lokalisiert, durch das hinzufügen des lokalen Kontextes zu dem Namen der Funktion (Beispiel: das heißt „my.calc“ wird zu „kl.my.calc“). Wenn die Funktion nicht im aktuellen Kontext gefunden werden kann, wird der unmodifizierte name verwendet (d.h. „my.calc“).

```
PROCEDURE cexecute(name: STRING)
```

Startet eine benutzerdefinierte Funktion. Anhand des Namens der Funktion wird der lokale Kontext gesetzt (Beispiel: Aus „kl.my.calc“ wird der lokale Kontext „kl“ abgeleitet).

```
PROCEDURE abort()
```

Bricht die Ausführung der Benutzerprogrammierung ab.

```
PROCEDURE user_abort()
```

Bricht die Ausführung der Benutzerprogrammierung ab.

```
PROCEDURE utilStop(s: STRING)
```

Bricht die Ausführung der Benutzerprogrammierung mit angegebener Fehlermeldung ab.

2.7 CSV-Logs

2.7.1 Methoden

```
FUNCTION LogRegister(SourceName, SlotName: STRING; Headers: ARGUMENTS):
INTEGER
```

Registriert und aktiviert ein neues CSV-Log mit angegebenem Namen und Spaltenbezeichnungen.

```
PROCEDURE LogAdd(SlotID: INTEGER; COLUMNS: ARGUMENTS)
```

Fügt einem CSV-Log eine neue Datenzeile hinzu.

```
PROCEDURE write(ARGS: ARGUMENTS)
```

Schreibt Daten in das Standard-ALM.IT-Log.

```
PROCEDURE writeln(ARGS: ARGUMENTS)
```

Schreibt Daten sowie einen Zeilenumbruch in das Standard-ALM.IT-Log.

2.8 Ltab

2.8.1 Methoden

```
PROCEDURE ltabinit(VAR lt: LTAB; sTableName: STRING; iRows, iCols: INTEGER)
```

Erzeugt eine neue Ltab mit angegebenen Namen und Tabellengröße.

```
FUNCTION ltabgetname(lt: LTAB): STRING
```

Gibt den Namen der übergebenen Ltab zurück.

```
FUNCTION ltabgetnumrow(lt: LTAB): INTEGER
```

Gibt die Anzahl der Zeilen der übergebenen Ltab zurück.

```
FUNCTION ltabgetnumcol(lt: LTAB): INTEGER
```

Gibt die Anzahl der Spalten der übergebenen Ltab zurück.

```
FUNCTION ltabgetheadnum(lt: LTAB; iCol: INTEGER): ANY
```

Gibt den Spalten-Header mit angegebenen Spaltenindex der übergebenen Ltab zurück.

```
FUNCTION ltabsetheadnum(lt: LTAB; AnyPar: ANY; iCol: INTEGER): BOOLEAN
```

Setzt den Spalten-Header am angegebenen Spaltenindex der übergebenen Ltab.

```
FUNCTION ltabgetcolnum(lt: LTAB; sHeadName: STRING): INTEGER
```

Gibt den Index der Spalte mit angebenem Namen der übergebenen Ltab zurück.

```
FUNCTION ltabgetheadname(lt: LTAB; sHeadName: STRING): ANY
```

Gibt den Header zum angegebenen Spaltennamen der übergebenen Ltab zurück.

```
FUNCTION ltabsetheadname(lt: LTAB; AnyPar: ANY; sHeadName: STRING): BOOLEAN
```

Setzt den Spalten-Header am angegebenen Spaltenname der übergebenen Ltab.

```
FUNCTION ltabgetrcn(lt: LTAB; iRow: INTEGER; sName: STRING): ANY
```

Gibt den Inhalt der übergebenen Ltab aus der angebenenen Zeile und Spaltenname zurück.

```
FUNCTION ltabsetrcn(lt: LTAB; a: ANY; iRow: INTEGER; sName: STRING):
BOOLEAN
```

Modifiziert den Inhalt der übergebenen Ltab an der angebenenen Zeile und Spaltenname.

```
FUNCTION ltabgetrc(lt: LTAB; iRow, iCol: INTEGER): ANY
```

Gibt den Inhalt der übergebenen Ltab aus der angebenenen Zeile und Spalte zurück.

```
FUNCTION ltabsetrc(lt: LTAB; a: ANY; iRow, iCol: INTEGER): BOOLEAN
```

Modifiziert den Inhalt der übergebenen Ltab an der angebenenen Zeile und Spalte.

```
FUNCTION ltabgetattr(lt: LTAB; sAttrName: STRING): ANY
```

Gibt das angegebene Attribut der übergebenen Ltab zurück.

```
FUNCTION ltabsetattr(lt: LTAB; sAttrName: STRING; AttrAny: ANY): BOOLEAN
```

Modifiziert das angegebene Attribut der übergebenen Ltab.

```
FUNCTION ltabremoveattr(lt: LTAB; sAttrName: STRING): BOOLEAN
```

Entfernt das angegebene Attribut aus der übergebenen Ltab.

```
FUNCTION ltabgetfun(lt: LTAB; sFunName: STRING): ANY
```

Gibt die angegebene Funktion der übergebenen Ltab zurück.

```
FUNCTION ltabsetfun(lt: LTAB; sFunName: STRING; FunAny: ANY): BOOLEAN
```

Modifiziert die angegebene Funktion der übergebenen Ltab.

```
FUNCTION ltabremovefun(lt: LTAB; sFunName: STRING): BOOLEAN
```

Entfernt die angegebene Funktion aus der übergebenen Ltab.

```
FUNCTION ltabgetattrlist(lt: LTAB): STRINGLIST
```

Gibt eine Stringlist mit den Namen der Attribute der übergebenen Ltab zurück.

```
FUNCTION ltabgetfunlist(lt: LTAB): STRINGLIST
```

Gibt eine Stringlist mit den Namen der Funktionen der übergebenen Ltab zurück.

```
FUNCTION ltabinsertcol(lt: LTAB; iCol: INTEGER; AnyHeader: ANY): INTEGER
```

Fügt in der übergebenen Ltab am angegebenen Spaltenindex eine neue Spalte hinzu.

```
FUNCTION ltabappendcol(lt: LTAB; AnyHeader: ANY): INTEGER
```

Fügt in der übergebenen Ltab eine neue Spalte an.

```
FUNCTION ltabremovecolnum(lt: LTAB; iCol: INTEGER): BOOLEAN
```

Entfernt die angegebene Spalte aus der übergebenen Ltab.

```
FUNCTION ltabremovecolname(lt: LTAB; sColName: STRING): BOOLEAN
```

Entfernt die angegebene Spalte aus der übergebenen Ltab.

```
FUNCTION ltabinsertrow(lt: LTAB; iRow: INTEGER): INTEGER
```

Fügt in die übergebene Ltab eine Zeile an angegebenem Zeilenindex ein.

```
FUNCTION ltabappendrow(lt: LTAB): INTEGER
```

Fügt an die übergebene Ltab eine Zeile an.

```
FUNCTION ltabremoverow(lt: LTAB; iRow: INTEGER): BOOLEAN
```

Entfernt aus der übergebenen Ltab die angegebene Zeile.

```
FUNCTION ltabremovelrow(lt: LTAB): BOOLEAN
```

Entfernt aus der übergebenen Ltab die letzte Zeile.

```
FUNCTION ltabload(VAR lt: LTAB; sTableName: STRING): BOOLEAN
```

Lädt die Ltab mit angegebenen Namen aus den gespeicherten Ergebnis-Ltabs.

```
FUNCTION ltabstore(lt: LTAB; sTableName: STRING): BOOLEAN
```

Speichert die Ltab unter angegebenen Namen bei den Ergebnis-Ltabs.

```
FUNCTION ltabgetheaders(lt: LTAB): STRINGLIST
```

Gibt eine Stringlist mit den Spaltennamen der übergebenen Ltab zurück.

```
FUNCTION ltabcompl(sTableName: STRING): STRING
```

Gibt den vollständigen Dateipfad für den angegebenen Tabellennamen zurück.

2.9 Mathematische Funktionen

2.9.1 Methoden

```
FUNCTION abs(d: REAL): REAL
```

Gibt den Absolutbetrag einer angegebenen Gleitkommazahl zurück.

```
FUNCTION ceil(d: REAL): REAL
```

Gibt den kleinsten ganzzahligen Wert zurück, der größer oder gleich der angegebenen Gleitkommazahl ist.

```
FUNCTION cos(d: REAL): REAL
```

Gibt den Kosinus des angegebenen Winkels (in Bogenmaß) zurück.

```
FUNCTION sin(d: REAL): REAL
```

Gibt den Sinus des angegebenen Winkels (in Bogenmaß) zurück.

```
FUNCTION floor(d: REAL): REAL
```

Gibt die größte Ganzzahl zurück, die kleiner oder gleich der angegebenen Gleitkommazahl ist.

```
FUNCTION power(d, f: REAL): REAL
```

Potenziert eine angegebene Zahl d mit dem angegebenen Exponenten f.

```
FUNCTION intpower(d: REAL; n: INTEGER): REAL
```

Potenziert eine angegebene Zahl d mit dem angegebenen Ganzzahl Exponenten d.

```
FUNCTION exp(d: REAL): REAL
```

Gibt den Wert von e hoch angegebenem Exponenten zurück.

```
FUNCTION log10(d: REAL): REAL
```

Gibt den Logarithmus einer angegebenen Zahl zur Basis 10 zurück.

```
FUNCTION max(a, b: REAL): REAL
```

Gibt die Größte zweier Zahlen zurück.

```
FUNCTION intmax(a, b: INTEGER): INTEGER
```

Gibt die Größte zweier Ganzzahlen zurück.

```
FUNCTION min(a, b: REAL): REAL
```

Gibt die Kleinste zweier Zahlen zurück.

```
FUNCTION intmin(a, b: INTEGER): INTEGER
```

Gibt die Kleinste zweier Ganzzahlen zurück.

```
FUNCTION logn(d, base: REAL): REAL
```

Gibt den Logarithmus zur Basis base der angegebenen Zahl d zurück.

```
FUNCTION round(d: REAL): INTEGER
```

Rundet die Zahl d kaufmännisch und gibt diese Zahl zurück.

```
FUNCTION ln(d: REAL): REAL
```

Gibt den natürlichen Logarithmus (zur Basis e) der angegebenen Zahl zurück.

```
FUNCTION arctan(d: REAL): REAL
```

Gibt einen Winkel zurück, dessen Tangens die angegebene Zahl ist.

```
FUNCTION sqr(d: REAL): REAL
```

Gibt das Quadrat einer übergebenen Gleitkommazahl zurück.

```
FUNCTION sqrt(d: REAL): REAL
```

Gibt die Quadratwurzel einer angegebenen Zahl zurück.

```
FUNCTION frac(d: REAL): REAL
```

Die Funktion Frac gibt den Nachkommaanteil einer reellen Zahl zurück.

```
FUNCTION intpart(d: REAL): REAL
```

Gibt die größte Ganzzahl zurück, die kleiner oder gleich der angegebenen Dezimalzahl ist.

```
FUNCTION trunc(d: REAL): INTEGER
```

Gibt den ganzzahligen Wert von d zurück.

```
PROCEDURE randomize(n: INTEGER)
```

Setzt den Seed des Random-Genrators auf n.

```
FUNCTION randu(): REAL
```

Gibt einen zufälligen Real Wert zwischen 0 und 1 zurück.

```
FUNCTION randg(dMean, dStdDev: REAL): REAL
```

Gibt einen mit angegebenen Parametern normalverteilten Zufallswert zurück.

2.10 Memory Hub

2.10.1 Methoden

```
PROCEDURE setint(name: STRING; value: INTEGER)
```

Diese Funktion versucht, ein Attribut vom Typ integer (value) unter dem angegebenen Schlüssel (name) im Mhub einzufügen. Bestehende Werte werden dabei gelöscht.

```
FUNCTION getint(name: STRING): INTEGER
```

Diese Funktion gibt ein Attribut vom Typ Integer mit dem Namen (name) von MHUB zurück. Wenn kein Attribut mit diesem Namen im Mhub existiert, wird eine Exception ausgelöst.

```
PROCEDURE setreal(name: STRING; value: REAL)
```

Diese Funktion versucht, ein Attribut vom Typ Real (value) unter dem angegebenen Schlüssel (name) im Mhub einzufügen. Bestehende Werte werden dabei gelöscht.

```
FUNCTION getreal(name: STRING): REAL
```

Diese Funktion gibt ein Attribut vom Typ Real mit dem Namen (name) von MHUB zurück. Wenn kein Attribut mit diesem Namen im Mhub existiert, wird eine Exception ausgelöst.

```
PROCEDURE setany(name: STRING; value: ANY)
```

Diese Funktion versucht, ein Attribut vom Typ Any (value) unter dem angegebenen Schlüssel (name) im Mhub einzufügen. Bestehende Werte werden dabei gelöscht.

```
FUNCTION getany(name: STRING): ANY
```

Diese Funktion gibt ein Attribut vom Typ Any mit dem Namen (name) von MHUB zurück. Wenn kein Attribut mit diesem Namen im Mhub existiert, wird eine Exception ausgelöst.

```
PROCEDURE setstr(name, value: STRING)
```

Diese Funktion versucht, ein Attribut vom Typ String (value) unter dem angegebenen Schlüssel (name) im Mhub einzufügen. Bestehende Werte werden dabei gelöscht.

```
FUNCTION getstr(name: STRING): STRING
```

Diese Funktion gibt ein Attribut vom Typ String mit dem Namen (name) von MHUB zurück. Wenn kein Attribut mit diesem Namen im Mhub existiert, wird eine Exception ausgelöst.

```
PROCEDURE insertltab(name: STRING; mInsLTab: LTAB)
```

Diese Funktion versucht, ein Attribut vom Typ Ltab (value) unter dem angegebenen Schlüssel (name) im Mhub einzufügen. Bestehende Werte werden dabei gelöscht.

```
PROCEDURE getltab(name: STRING; VAR mInsLTab: LTAB)
```

Diese Methode liest ein Attribut vom Typ Ltab mit dem Namen (name) von MHUB aus. Wenn kein Attribut mit diesem Namen im Mhub existiert, wird eine Exception ausgelöst.

```
PROCEDURE insertrset(name: STRING; mInsRSet: RSET)
```

Diese Funktion versucht, ein Attribut vom Typ Rset (value) unter dem angegebenen Schlüssel (name) im Mhub einzufügen. Bestehende Werte werden dabei gelöscht.

```
PROCEDURE getrset(name: STRING; VAR mInsRSet: RSET)
```

Diese Methode liest ein Attribut vom Typ Rset mit dem Namen (name) von MHUB aus. Wenn kein Attribut mit diesem Namen im Mhub existiert, wird eine Exception ausgelöst.

```
PROCEDURE insertdmat(name: STRING; VAR dm: DMAT)
```

Diese Funktion versucht, ein Attribut vom Typ Dmat (value) unter dem angegebenen Schlüssel (name) im Mhub einzufügen. Bestehende Werte werden dabei gelöscht.

```
PROCEDURE getdmat(name: STRING; VAR dm: DMAT)
```

Diese Methode liest ein Attribut vom Typ Dmat mit dem Namen (name) von MHUB aus. Wenn kein Attribut mit diesem Namen im Mhub existiert, wird eine Exception ausgelöst.

```
PROCEDURE insertdvec(name: STRING; VAR dv: DVEC)
```

Diese Funktion versucht, ein Attribut vom Typ Dvec (value) unter dem angegebenen Schlüssel (name) im Mhub einzufügen. Bestehende Werte werden dabei gelöscht.

```
PROCEDURE getdvec(name: STRING; VAR dv: DVEC)
```

Diese Methode liest ein Attribut vom Typ Dvec mit dem Namen (name) von MHUB aus. Wenn kein Attribut mit diesem Namen im Mhub existiert, wird eine Exception ausgelöst.

```
PROCEDURE setsvec(name: STRING; sv: SVEC)
```

Diese Funktion versucht, ein Attribut vom Typ Svec (value) unter dem angegebenen Schlüssel (name) im Mhub einzufügen. Bestehende Werte werden dabei gelöscht.

```
PROCEDURE getsvec(name: STRING; VAR sv: SVEC)
```

Diese Methode liest ein Attribut vom Typ Svec mit dem Namen (name) von MHUB aus. Wenn kein Attribut mit diesem Namen im Mhub existiert, wird eine Exception ausgelöst.

```
FUNCTION hasany(name: STRING): BOOLEAN
```

Prüft, ob ein Any mit dem Namen (name) im Mhub existiert.

```
FUNCTION hasint(name: STRING): BOOLEAN
```

Prüft, ob ein Integer mit dem Namen (name) im Mhub existiert.

```
FUNCTION hasreal(name: STRING): BOOLEAN
```

Prüft, ob ein Real mit dem Namen (name) im Mhub existiert.

```
FUNCTION hasdmat(name: STRING): BOOLEAN
```

Prüft, ob eine Dmat mit dem Namen (name) im Mhub existiert.

```
FUNCTION hasdvec(name: STRING): BOOLEAN
```

Prüft, ob ein Dvec mit dem Namen (name) im Mhub existiert.

```
FUNCTION hasltab(name: STRING): BOOLEAN
```

Prüft, ob eine Ltab mit dem Namen (name) im Mhub existiert.

```
FUNCTION hasrset(name: STRING): BOOLEAN
```

Prüft, ob ein Rset mit dem Namen (name) im Mhub existiert.

```
FUNCTION hassvec(name: STRING): BOOLEAN
```

Prüft, ob ein Svec mit dem Namen (name) im Mhub existiert.

```
FUNCTION hasstr(name: STRING): BOOLEAN
```

Prüft, ob ein String mit dem Namen (name) im Mhub existiert.

2.11 Pivotwürfel

2.11.1 Methoden

```
PROCEDURE ClearPivotTranslations()
```

Löscht alle registrierten Pivotname-Übersetzungen.

```
PROCEDURE AddPivotTranslation(SourceName, DestName: STRING)
```

Registriert eine neue Pivotname-Übersetzung.

```
PROCEDURE ClearPivotKey()
```

Löscht alle Ausprägungen aus dem aktuellen Pivot-Schlüssel.

```
PROCEDURE AddPivotKey(CategoryName, CategoryValue: STRING)
```

Fügt zum aktuellen Pivot-Schlüssel eine Kategorie-Ausprägung hinzu.

```
PROCEDURE AddPivotKeyUntranslated(CategoryName, CategoryValue: STRING)
```

Fügt zum aktuellen Pivot-Schlüssel eine Kategorie-Ausprägung hinzu, ohne eine automatische Übersetzung auf Standard-Kategorieausprägungen zu versuchen.

```
PROCEDURE AddPivotValue(PivotName: STRING; ProjYear: INTEGER; Value: REAL)
```

Addiert in den Pivotwürfel einen Wert mit dem aktuellen Pivot-Schlüssel.

```
PROCEDURE AddPivotValueUntranslated(PivotName: STRING; ProjYear: INTEGER;
Value: REAL)
```

Addiert in den Pivotwürfel einen Wert mit dem aktuellen Pivot-Schlüssel, ohne eine automatische Übersetzung auf Standard-Pivotart zu versuchen.

```
PROCEDURE AddPivotValueMax(PivotName: STRING; ProjYear: INTEGER; Value:
REAL)
```

Maximiert in den Pivotwürfel einen Wert mit dem aktuellen Pivot-Schlüssel.

```
PROCEDURE AddPivotValueMaxUntranslated(PivotName: STRING; ProjYear:
INTEGER; Value: REAL)
```

Maximiert in den Pivotwürfel einen Wert mit dem aktuellen Pivot-Schlüssel, ohne eine automatische Übersetzung auf Standard-Pivotart zu versuchen.

2.11.2 Standardkategorien

Kategoriename	Beschreibung
PivotName	Art
Order	Ordnung
TariffType	Tarifart
TariffName	Tarifname
Gender	Geschlecht
Group	Gruppe
Status	Status
Segment	Segment
INBV_Segment	INBV Segment
ActuarialInterestRate	Rechnungszins
ArtDesGeschaefts	Art des Geschäfts

Zahlungsstatus	Zahlungsstatus
BSM_LoB	BSM LoB (LoB für klassischen Anteil bei VT Neu Initialwerte)
BSM_LoB_Fonds	BSM LoB für Fondsanteil bei VT Neu Initialwerte
AccountingSeries	Abrechnungsverband
Bewegungsart	Bewegungsart
Produktart	Produktart
BSM_Modelpoint	Modelpoint ID für VT Neu / VT Neu Initialwerte
BSM_UebNueb	Überschussberechtigung bei VT Neu Initialwerte
BSM_Bestandsgruppe_Rentenphase	Bestandsgruppe für die Rentenphase bei VT Neu Initialwerte
BSM_Ueberschussgruppe	Überschussgruppe

2.11.3 Standardausprägungen der Kategorie Art (PivotName)

Kategorieausprägung	Beschreibung
BSM_RZ	Rechnungszinsgeneration
BSM_Kosten	Kosten
BSM_Praemien	Prämien
BSM_LstgTod	Leistungen beim Tod
BSM_LstgKapAbf	Kapitalabfindungen (nur Rentenversicherung)
BSM_LstgSonstErl	sonstige Erlebensfallleistungen
BSM_Rueckkauf	Rückkauf
BSM_Risikoergebnis	Risikoergebnis
BSM_UebrigesErgebnis	Übriges Ergebnis
BSM_Zinsratenzuschlag	Zinsratenzuschlag
BSM_Zinsaufwand	Rechnungsmäßiger Zinsaufwand
BSM_HGB_DRSt	HGB DRSt inkl. Ansammlungsguthaben
BSM_CF_RV	CF EVU->RVU
BSM_FLV_KostenRentenbezug	Kosten (Rentenbezug)
BSM_FLV_PraemienRentenbezug	Prämien (Rentenbezug)
BSM_FLV_LeistungBeiTodRentenbezug	Leistungen beim Tod (Rentenbezug)
BSM_FLV_SonstigeErlebensfallleistungRentenbezug	Sonstige Erlebensfallleistungen (Rentenbezug)
BSM_FLV_RisikoergebnisKlassik	Riskoergebnis klassische Renten (Rentenbezug)

BSM_FLV_UebrigesErgebnisKlassik	Übriges Ergebnis klassische Renten (Rentenbezug)
BSM_FLV_RechnungsmaessigerZinsaufwand	Rechnungsmäßiger Zinsaufwand (Rentenbezug)
BSM_FLV_HGBDrSt	HGB DRSt (Rentenbezug)
BSM_FLV_KostenAufschub	Kosten VU (Aufschubzeit)
BSM_FLV_PraemienAufschub	Prämien (Aufschubzeit)
BSM_FLV_LeistungBeiTodAufschub	Leistungen bei Tod (Aufschubzeit)
BSM_FLV_SonstigeErlebensfallleistungAufschub	Sonstige Erlebensfallleistungen (Aufschubzeit)
BSM_FLV_Kapitalabfindung	Kapitalabfindungen (Aufschubzeit)
BSM_FLV_Rueckkauf	Rückkauf (Aufschubzeit)
BSM_FLV_RisikoergebnisFLV	Risikoergebnis (Aufschubzeit)
BSM_FLV_UebrigesErgebnisFLV	Übriges Ergebnis (Aufschubzeit)
BSM_FLV_KostenFondsverwaltung	Kosten Fondsverwaltung
BSM_FLV_AktienanteilFonds	Aktienanteil im Fond
BSM_FLV_ImmobilienanteilFonds	Immobilienanteil im Fond
BSM_FLV_FI_AnteilFonds	FI-Anteil im Fond
BSM_FLV_Fondsentwicklung	Fondsentwicklung
BSM_SUEAF_Startwert	SÜAF Startwert
BSM_ZZR_Startwert	ZZR Startwert
BSM_ZZR_Aufwand	ZZR Aufwand je Basispunkt
BSM_Korrekturterm_ZZR	Korrekturterm ZZR wegen Rechnungsgrundlagen
BSM_Startwert_Referenzzins	Startwert Referenzzins
BSM_VToStress_ZZR_A	BSM VT o. Stress: ZZR zu Referenzzinssatz A
BSM_VToStress_ZZR_B	BSM VT o. Stress: ZZR zu Referenzzinssatz B
BSM_VToStress_ZinsDiff_A	BSM VT o. Stress: Zinsdifferenz Referenzzinssatz A
BSM_VToStress_ZinsDiff_B	BSM VT o. Stress: Zinsdifferenz Referenzzinssatz B
BSM_VToStress_FLVZZRAufwand	BSM VT o. Stress: FLV ZZR Aufwand je Basispunkt je Euro Fondsguthaben
BSM_VToStress_FLVZZRStartKonventionell	BSM VT o. Stress: FLV ZZR Startwert, konventionell bedeckt
BSM_VToStress_FLVZZRStartFonds	BSM VT o. Stress: FLV ZZR Startwert, fondsbedeckt

BSM_VTNeu_K_Stueck_T	BSM VT Neu: Erwartete volumenunabhängige Kosten VU (2. Ordnung)
BSM_VTNeu_i_r_Rentenbezug	BSM VT Neu: i_r_Rentenbezug
BSM_VTNeu_K_1O_Vol_Stueck	BSM VT Neu: kalkulatorische volumenunabhängige Kosten VU (1. Ordnung)
BSM_VTNeu_k_Faktor_Vol_SV_t	BSM VT Neu: volumenabhängige Kosten VU je € Sicherungsvermögen (2. Ordnung)
BSM_VTNeu_k_Faktor_Vol_WSF_ext	BSM VT Neu: Erwartete weitere externe Kosten der Wertsicherung in % Wertsicherungsfonds
BSM_VTNeu_k_Faktor_Vol_F	BSM VT Neu: volumenabhängige Kosten VU je € Wertsicherungs-Fonds (2. Ordnung)
BSM_VTNeu_k_Faktor_Vol_fF_t	BSM VT Neu: volumenabhängige Kosten VU je € freien Fonds (2. Ordnung)
BSM_VTNeu_k_Faktor_1O_Vol_SV	BSM VT Neu: volumenabhängige Kosten VU je € Sicherungsvermögen (1. Ordnung)
BSM_VTNeu_k_Faktor_1O_Vol_SV_UEB	BSM VT Neu: Kalkulatorische bedingte Kosten in % des Sicherungsvermögens pro Jahr, sofern eine Überschussbeteiligung gewährt wird
BSM_VTNeu_k_Faktor_1O_Vol_F	BSM VT Neu: volumenabhängige Kosten VU je € Wertsicherungs-Fonds (1. Ordnung)
BSM_VTNeu_k_Faktor_1O_Vol_fF	BSM VT Neu: volumenabhängige Kosten VU je € freien Fonds (1. Ordnung)
BSM_VTNeu_k_KAG_Vol_WSF_Faktor	BSM VT Neu: Kosten der Fondsverwaltung in % des (Wertsicherungs-)fonds pro Jahr
BSM_VTNeu_k_KAG_Vol_fF_Faktor	BSM VT Neu: Kosten der Fondsverwaltung in % des freien Fonds pro Jahr
BSM_VTNeu_VolumenUnabhSaldo_EVU_RVU	BSM VT Neu: volumenunabhängiger Saldo EVU -> RVU
BSM_VTNeu_VolumenAbhSaldo_EVU_RVU _SichVerm_WSF_freieFonds	BSM VT Neu: volumenabhängige Saldos EVU -> RVU Sicherungsvermögen, Wertsicherung, freie Fonds
BSM_VTNeu_B_t	BSM VT Neu: Prämien (ohne Abschlusskostenanteile aber inkl. festg. Überschussbeteil. Aus der RfB der

	ersten Jahre) - Prämien (Brutto, in den Fonds), Aufschubzeit, Euro - Beitrag B (Anfang des Jahres)
BSM_VTNeu_tfl_absolut_t	BSM VT Neu: volumenunabhängige Todesfallleistung Anteil je € des Vertragsguthabens (damit unabhängig von der Kapitalmarktentwicklung)
BSM_VTNeu_tfl_Faktor	BSM VT Neu: volumenabhängige (Mindest)-TFL
BSM_VTNeu_RE_det_t	BSM VT Neu: deterministischer Verlauf des Risikoergebnisses (optional - relevant bei TFL Berechnung mit Approximation des riskierten Kapitals)
BSM_VTNeu_RK_det_t	BSM VT Neu: deterministischer Verlauf des riskierten Kapitals (optional - relevant bei TFL Berechnung mit Approximation des riskierten Kapitals)
BSM_VTNeu_e_t	BSM VT Neu: Ausgezahlte Erlebensfallleistung je Euro Vertragsguthaben und Jahr
BSM_VTNeu_Rentenleistung _Rentenbezug_Mindestgarantie	BSM VT Neu: Rentencashflow / Leistung Rentenbezug auf Basis der Mindestgarantie
BSM_VTNeu_Rentenleistung_Interpolation1	BSM VT Neu: Rentencashflow / Leistung pro € Guthaben entsprechend Interpolationsstützstelle 1
BSM_VTNeu_Rentenleistung_Interpolation2	BSM VT Neu: Rentencashflow / Rentenleistung pro € Guthaben entsprechend Interpolationsstützstelle 2
BSM_VTNeu_s_t	BSM VT Neu: Stornowahrscheinlichkeit je Euro Vertragsguthaben und Jahr
BSM_VTNeu_kw_t	BSM VT Neu: für Kapitalwahl relevante Erlebnisfallleistung je € Vertragsguthaben
BSM_VTNeu_DR_t_Plus	BSM VT Neu: HGB DRSt inkl. Ansammlungsguthaben und festgelegte RfB ohne ZZR, abzüglich Aktivierte Ansprüche
BSM_VTNeu_Floor_t_Plus	BSM VT Neu: Floor am Ende des Zeitschritts t
BSM_VTNeu_i_g	BSM VT Neu: Garantiezins
BSM_VTNeu_Gar_Primaer_t	BSM VT Neu: bestehende endfällige Primärgarantie
BSM_VTNeu_niveau_HSt_Sich	BSM VT Neu: Niveau der Höch-

	standssicherung: Parameters zwischen 0 und 1, der Niveau der „Höchststandsicherung“ beschreibt der eine Sicherung auf niedrigerem Niveau ermöglicht
BSM_VTNeu_DR_ZG_t_Plus	BSM VT Neu: Bilanzieller Deckungsrückstellungsverlauf für 1 € endfälliger Zusatzgarantie Eintrittswahrscheinlichkeiten bereits gewichtet
BSM_VTNeu_Floor_ZG_t_Plus	BSM VT Neu: Floor_ZG Floorverlauf für 1 € endfälliger Zusatzgarantie mit Eintrittswahrscheinlichkeiten bereits gewichtet
BSM_VTNeu_u_b	BSM VT Neu: u(b), UbAufteilung_Hyb, u_factor_t unternehmensindividuell festzulegender Faktor zur Aufteilung der Deklaration der Zinsüberschüsse
BSM_VTNeu_AnteileGarantiefondsAssetA	BSM VT Neu: Aktienanteil im Garantiefonds in % (Dient der Modellierung der Entwicklung des Garantiefonds)
BSM_VTNeu_AnteileGarantiefondsAssetB	BSM VT Neu: FI-anteil im Garantiefonds in % (Dient der Modellierung der Entwicklung des Garantiefonds)
BSM_VTNeu_AnteileGarantiefondsAssetC	BSM VT Neu: Nummer des zusätzlichen Assets (siehe zu den ESG Daten ergänzte Assets)
BSM_VTNeu_AnteileFreierFondsAssetA	BSM VT Neu: Aktienanteil im freien Fonds (Dient der Modellierung der Entwicklung des freien Fonds) in %
BSM_VTNeu_AnteileFreierFondsAssetB	BSM VT Neu: Immobilienanteil im freien Fonds (Dient der Modellierung der Entwicklung des freien Fonds) in %
BSM_VTNeu_AnteileFreierFondsAssetC	BSM VT Neu: FI-anteil im freien Fonds (Dient der Modellierung der Entwicklung des freien Fonds) in %
BSM_VTNeu_qmax_Fonds	BSM VT Neu: Maximale Fondsquote
BSM_VTNeu_qmax_fFonds	BSM VT Neu: maximale Quote freier Fonds
BSM_VTNeu_q_1O_t	BSM VT Neu: q^1O Abgangswahrscheinlichkeit aufgrund von Tod des Modellpoints 1. Ordnung (kalkulatorische Eintrittswahrscheinlichkeit für Todesfallleistung)
BSM_VTNeu_q_2O_t	BSM VT Neu: q^2O Abgangswahrscheinlichkeit aufgrund von Tod des Modellpoints 2. Ordnung (Eintritts-

	wahrscheinlichkeit für Todesfallleistung)
BSM_VTNeu_Kosten_Rentenbezug	BSM VT Neu: Kosten (Rentenbezug), Euro
BSM_VTNeu_Beitrag_Rente_Det_DHP	BSM VT Neu: Beitrag_Rente_Det_DHP Prämien = Verrentendes Kapital (aus Ende Aufschub des Fonds), Euro
BSM_VTNeu_Leistungen_Beim_Tod_Rentenbezug	BSM VT Neu: Leistungen beim Tod (in Rentenbezug), Euro
BSM_VTNeu_Sonstige_Erlebensfallleistungen_Rentenbezug	BSM VT Neu: Sonstige Erlebensfallleistungen (laufende Renten), Rentenbezug, Euro
BSM_VTNeu_RisikoergebnisKlassik_Rentenbezug	BSM VT Neu: Riskoergebnis klassische Renten, Rentenbezug, Euro
BSM_VTNeu_UebrigesErgebnisKlassik_Rentenbezug	BSM VT Neu: Übriges Ergebnis Klassische Renten, Rentenbezug, Euro
BSM_VTNeu_Rechnungzinsaufwand_Rentenbezug	BSM VT Neu: Rechnungsmäßiger Zinsaufwand, Rentenbezug, Euro
BSM_VTNeu_HGBDRSt_Rentenbezug	BSM VT Neu: HGB DRSt inkl. Ansammlungsguthaben und festgelegte RfB ohne ZZR, Rentenbezug, Euro
BSM_VTNeu_rvt	BSM VT Neu: Anteil (rvt) Rückversicherer Aufwand Zusatzreserve
BSM_VTNeu_wvRvt	BSM VT Neu: Mindestwertverlust (wvRvt) für Leistung RV
BSM_VTNeu_rateRvt	BSM VT Neu: Prämienrate RV (rateRvt)
BSM_VTNeuOStress_SUEAF_Startwert	BSM VT Neu o. Stress: SÜAF nur als Startwert zum Stichtag (Zeitpunkt 0)
BSM_VTNeuOStress_auf_pro_BP_t	BSM VT Neu o. Stress: ZZR Aufwand je Basispunkt für neue Produkte
BSM_VTNeuOStress_auf_pro_BP_VG_t	BSM VT Neu o. Stress: Neue Produkte ZZR3 Aufwand je Basispunkt je Euro Vertragsguthaben
BSM_VTNeuOStress_ZZR_3_konvBed_t0_Plus	BSM VT Neu o. Stress: Neue Produkte ZZR3 Startwert, konventionell bedeckt
BSM_VTNeuInitial_i_r	BSM VT Neu Initialwerte: Rechnungszinsgeneration / tariflicher Rechnungszins
BSM_VTNeuInitial_FloorBerechnet	BSM VT Neu Initialwerte: Schalter: Floor aus Input (0) oder berechnet (1)

BSM_VTNeuInitial_ClusterID	BSM VT Neu Initialwerte: Cluster_ID für die Rückaufteilung der Aggregierten Werte aus der Deklaration auf die einzelnen Modelpoints
BSM_VTNeuInitial_wv	BSM VT Neu Initialwerte: Garantiefonds / Wertsicherungsfonds - Maximaler Wertverlust in %
BSM_VTNeuInitial_I_auffuellbetrag_auf_Floor	BSM VT Neu Initialwerte: 0 oder 1; 1 bedeutet, dass abhängig vom Produkt, zum Ende einer Periode zum Ausgleich einer Unterdeckung des Floors das Guthaben bis zum Floor aufgefüllt wird.
BSM_VTNeuInitial_I_nur_fondsinterneAbsicherung	BSM VT Neu Initialwerte: 0 oder 1; 1 bedeutet, dass die Wertsicherung des Fondsvermögens ausschließlich innerhalb des Wertsicherungsfonds erfolgt.
BSM_VTNeuInitial_iExterneAbsicherungFonds	BSM VT Neu Initialwerte: iExterneAbsicherungFonds [0 = nein, 1 = ja], wenn "ja" ist zusätzlich ext. Abs.-Niveau zu befüllen (Spalte Z)
BSM_VTNeuInitial_SichV_t0_Plus	BSM VT Neu Initialwerte: Startguthaben Sicherungsvermögen
BSM_VTNeuInitial_Fonds_t0_Plus	BSM VT Neu Initialwerte: Startguthaben Wertsicherungsfonds
BSM_VTNeuInitial_fFonds_t0_Plus	BSM VT Neu Initialwerte: Startguthaben Freier Fonds
BSM_VTNeuInitial_zr_t0_Plus	BSM VT Neu Initialwerte: Startwert der Zusatzreserve
BSM_VTNeuInitial_ZZR_fondsBed_t0_Plus	BSM VT Neu Initialwerte: Neue Produkte ZZRStartwert, fondsbedeckt
BSM_VTNeuInitial_ESGSpalteAssetA	BSM VT Neu Initialwerte: ESG Spalte Für Asset A
BSM_VTNeuInitial_ESGSpalteAssetB	BSM VT Neu Initialwerte: ESG Spalte Für Asset B
BSM_VTNeuInitial_ESGSpalteAssetC	BSM VT Neu Initialwerte: ESG Spalte Für Asset C
BSM_VTNeuInitial_ArtZufuehrungRfBUeberlauf	BSM VT Neu Initialwerte: Schalter, für Art der Zuführung eines RfB-Überlaufs, der einem SÜAF zugewiesen wird
BSM_VTNeuInitial_RechnungszinsRentenphase	BSM VT Neu Initialwerte: Rechnungszins Rentenphase
BSM_VTNeuInitial_ArtUmschichtung	BSM VT Neu Initialwerte: Art des Um-

	schichtungsalgorithmus
BSM_VTNeuInitial_SUEAF	BSM VT Neu Initialwerte: SÜAF in t = 0
BSM_VTNeuInitial_UeberfuehrungSUEAF	BSM VT Neu Initialwerte: Schalter mit dem die Art der Überführung eines SÜAF in die Rentenphase festgelegt wird
BSM_VTNeuInitial_WvExtern	BSM VT Neu Initialwerte: Wertverlust für externe Absicherung des Wertsicherungsfonds (wvExtern)

2.11.4 Standardausprägungen der Kategorie Art des Geschäfts (ArtDesGeschaefts)

Kategorieausprägung	Beschreibung
NachArtDerLeben	Nach Art der Lebensversicherung
NachArtDerKranken	Nach Art der Krankenversicherung
NachArtDerSchaden	Nach Art der Schadenversicherung

2.11.5 Standardausprägungen der Kategorie Abrechnungsverband (AccountingSeries)

Kategorieausprägung	Beschreibung
Altbestand	Altbestand
Neubestand	Neubestand

2.11.6 Standardausprägungen der Kategorie Überschussberechtigung (BSM_UebNueb)

Kategorieausprägung	Beschreibung
ueb	Überschussberechtigt
nueb	Nicht überschussberechtigt

2.12 Rset

2.12.1 Methoden

```
PROCEDURE rinit(VAR refRset: RSET; sTableName: STRING)
```

Erstellt ein neues Recordset mit angebenem Tabellennamen.

```
FUNCTION rgetname(rset: RSET): STRING
```

Gibt den Namen des übergebenen Recordsets zurück.

```
FUNCTION rgetnumrow(rset: RSET): INTEGER
```

Gibt die Anzahl der Zeilen des übergebenen Recordsets zurück.

```
FUNCTION rgetnumcol(rset: RSET): INTEGER
```

Gibt die Anzahl der Spalten des übergebenen Recordsets zurück.

```
FUNCTION rgetcolumn(rset: RSET; name: STRING): INTEGER
```

Gibt den Index der Spalte mit angegebenen Spaltennamen des übergebenen Recordsets zurück.

```
FUNCTION rgetcolname(rset: RSET; iCol: INTEGER): STRING
```

Gibt den Namen der Spalte mit angegebenem Spaltenindex des übergebenen Recordsets zurück.

```
FUNCTION rgetcoltype(rset: RSET; iCol: INTEGER): STRING
```

Gibt den Datentyp der Spalte mit angegebenem Spaltenindex des übergebenen Recordsets zurück (Mögliche Ausprägungen: Integer, Real, String, Boolean, Undefined).

```
FUNCTION rgetboolean(rset: RSET; iCol: INTEGER): BOOLEAN
```

Gibt den aktuellen Zeilen-Wert der Boolean-Spalte mit angegebenen Spaltenindex des übergebenen Recordsets zurück.

```
FUNCTION rsetboolean(rset: RSET; iCol: INTEGER; value: BOOLEAN): BOOLEAN
```

Setzt den aktuellen Zeilen-Wert der Boolean-Spalte mit angegebenen Spaltenindex des übergebenen Recordsets.

```
FUNCTION rgetreal(rset: RSET; iCol: INTEGER): REAL
```

Gibt den aktuellen Zeilen-Wert der Real-Spalte mit angegebenen Spaltenindex des übergebenen Recordsets zurück.

```
FUNCTION rsetreal(rset: RSET; iCol: INTEGER; value: REAL): BOOLEAN
```

Setzt den aktuellen Zeilen-Wert der Real-Spalte mit angegebenen Spaltenindex des übergebenen Recordsets.

```
FUNCTION rgetinteger(rset: RSET; iCol: INTEGER): INTEGER
```

Gibt den aktuellen Zeilen-Wert der Integer-Spalte mit angegebenen Spaltenindex des übergebenen Recordsets zurück.

```
FUNCTION rsetinteger(rset: RSET; iCol, value: INTEGER): BOOLEAN
```

Setzt den aktuellen Zeilen-Wert der Integer-Spalte mit angegebenen Spaltenindex des übergebenen Recordsets.

```
FUNCTION rgetstring(rset: RSET; iCol: INTEGER): STRING
```

Gibt den aktuellen Zeilen-Wert der String-Spalte mit angegebenen Spaltenindex des übergebenen Recordsets zurück.

```
FUNCTION rsetstring(rset: RSET; iCol: INTEGER; value: STRING): BOOLEAN
```

Setzt den aktuellen Zeilen-Wert der String-Spalte mit angegebenen Spaltenindex des übergebenen Recordsets.

```
FUNCTION rmovefirst(rset: RSET): BOOLEAN
```

Setzt die aktuelle Zeilenposition des übergebenen Recordsets auf die erste Zeile.

```
FUNCTION rmovenext(rset: RSET): BOOLEAN
```

Setzt die aktuelle Zeilenposition des übergebenen Recordsets auf die nächste Zeile.

```
FUNCTION rmovelast(rset: RSET): BOOLEAN
```

Setzt die aktuelle Zeilenposition des übergebenen Recordsets auf die letzte Zeile.

```
FUNCTION radvance(rset: RSET; rows: INTEGER): BOOLEAN
```

Verschiebt die aktuelle Zeilenposition des übergebenen Recordsets um die angebene Anzahl an Zeilen.

```
FUNCTION raddrow(rset: RSET): BOOLEAN
```

Fügt eine Zeile am Ende des übergebenen Recordsets hinzu.

```
FUNCTION rappend(rset, rsetAppend: RSET): BOOLEAN
```

Fügt die Zeilen eines übergebenen Recordsets dem angebenen Recordset an.

```
FUNCTION rbackup(rset: RSET; id: STRING): BOOLEAN
```

Speichert die Daten des übergebenen Recordsets in ein angegebenes Backup.

```
FUNCTION rrestore(rset: RSET; id: STRING): BOOLEAN
```

Ersetzt die aktuellen Daten des übergebenen Recordsets durch das angegebene Backup.

```
FUNCTION rstore(rset: RSET; sTableName: STRING): BOOLEAN
```

Speichert das RecordSet unter angegebenen Namen bei den Ergebnis-Recordsets.

```
FUNCTION rload(VAR rset: RSET; sTableName: STRING): BOOLEAN
```

Lädt das Recordset mit angegebenen Namen aus den gespeicherten Ergebnis-Recordsets.

```
FUNCTION raddcolinteger(rset: RSET; name: STRING): BOOLEAN
```

Fügt dem übergebenen Recordset eine neue Integer-Spalte hinzu.

```
FUNCTION raddcolboolean(rset: RSET; name: STRING): BOOLEAN
```

Fügt dem übergebenen Recordset eine neue Boolean-Spalte hinzu.

```
FUNCTION raddcolreal(rset: RSET; name: STRING): BOOLEAN
```

Fügt dem übergebenen Recordset eine neue Real-Spalte hinzu.

```
FUNCTION raddcolstring(rset: RSET; name: STRING): BOOLEAN
```

Fügt dem übergebenen Recordset eine neue String-Spalte hinzu.

```
FUNCTION raddcolstringsize(rset: RSET; name: STRING; size: INTEGER):
BOOLEAN
```

Fügt dem übergebenen Recordset eine neue String-Spalte mit angegebener Maximallänge hinzu.

```
FUNCTION rclear(rset: RSET): BOOLEAN
```

Entfernt alle Zeilen sowie alle Backups aus dem übergebenen Recordset.

```
FUNCTION rgetattr(rset: RSET; sAttrName: STRING): ANY
```

Gibt das Attribut mit dem Namen sAttrName von refRSet zurück.

```
FUNCTION rsetattr(rset: RSET; sAttrName: STRING; AttrAny: ANY): BOOLEAN
```

Setzt ein durch Namen spezifiziertes Attribut eines Recordsets auf einen übergebenen Wert.

```
FUNCTION rremoveattr(rset: RSET; sAttrName: STRING): BOOLEAN
```

Entfernt das Attribut mit dem Name sAttrName aus refRSet.

```
FUNCTION rgetattrlist(rset: RSET): STRINGLIST
```

Gibt eine Stringlist mit den Namen der im übergebenen Recordset vorhandenen Attribute zurück.

2.13 Strings

2.13.1 Methoden

```
FUNCTION stradd(s1, s2: STRING): STRING
```

Verkettet zwei angegebene Zeichenfolgen.

```
FUNCTION strgetsubstr(s: STRING; start, length: INTEGER): STRING
```

Gibt eine Teilzeichenfolge aus einer übergebenen Zeichenfolge zurück.

```
FUNCTION strfindchar(s, sc: STRING): INTEGER
```

Gibt den Index des ersten Vorkommens der Zeichenfolge sc in s wieder.

```
FUNCTION strfindstr(s, sc: STRING): INTEGER
```

Gibt den Index des ersten Vorkommens der Zeichenfolge sc in s wieder.

```
FUNCTION mkstr(s: STRING): STRING
```

Gibt den String s wieder.

```
FUNCTION strisempty(s: STRING): BOOLEAN
```

Gibt an, ob die angegebene Zeichenfolge leer ist.

```
FUNCTION strlen(s: STRING): INTEGER
```

Gibt die Anzahl der Zeichen in der angegebenen Zeichenfolge zurück.

```
FUNCTION upcase(s: STRING): STRING
```

Wandelt den übergebenen String in Großbuchstaben um und ihn zurück.

```
FUNCTION inttostr(value: INTEGER): STRING
```

Wandelt den Integer value in einen String um und gibt diesen zurück.

```
FUNCTION strtoint(value: STRING): INTEGER
```

Wandelt den String s in einen Integer um und gibt diesen zurück.

```
FUNCTION floattostr(value: REAL): STRING
```

Wandelt den Real-Wert d in einen String um und gibt diesen zurück.

```
FUNCTION strtofloat(value: STRING): REAL
```

Wandelt den String s in Real um und gibt diesen zurück.

2.14 Stringlist

2.14.1 Methoden

```
FUNCTION stringlistfind(sl: STRINGLIST; value: STRING): BOOLEAN
```

Überprüft, ob die übergebene Stringliste einen angebenen Eintrag enthält.

```
FUNCTION stringlistgetiter(sl: STRINGLIST): STRINGLISTITER
```

Gibt einen Stringlist-Iterator für die übergebene Stringliste zurück.

```
FUNCTION stringlistisitervalid(sl: STRINGLIST; sliter: STRINGLISTITER):
BOOLEAN
```

Prüft, ob es sich bei dem Iterator sliter um einen Iterator für die Stringliste sl handelt.

```
PROCEDURE stringlistiterstep(sliter: STRINGLISTITER)
```

Bewegt den Iterator eins weiter.

```
FUNCTION stringlistitergetitem(sliter: STRINGLISTITER): STRING
```

Gibt den aktuellen String auf dem sliter zeigt zurück.

2.15 Svec

2.15.1 Methoden

```
FUNCTION svecsize(sv: SVEC): INTEGER
```

Gibt die Länge von sv zurück.

```
PROCEDURE svecresize(VAR sv: SVEC; iNewSize: INTEGER)
```

Setzt sv auf die neue Größe iNewSize.

3. Transportfunktion

Für die Verknüpfung des ALM.IT Liability Vectors mit dem PLA.NET Leistungsspektrum kann eine Transport-Funktion im Liability Vector definiert werden.

3.1 Standard-Zuordnung zum Leistungsspektrum

Sofern die Semantik des Basis Liability Vectors nicht wesentlich verändert wurde, kann folgende Zuordnung zugrunde gelegt werden:

Liabilty Vector Spaltenname	Zugehörige Leistungsspektrumsgröße
Aktivierung	AusstehendeAbschlusskosten
AnsammlungsGuthaben	AnsammlungsguthabenEnde
BeitragsUebertrag	Beitragsübertrag
BilanzReserve	DeckungsrückstellungEnde nach Abzug von ZinsZusatzReserve, BilanzReserveNRes, Bonusreserve und Kapitalanlage_FV
Bonusreserve	DeckungsrückstellungBonusEnde
ZinsZusatzReserve	Zinszusatzreserve
BilanzReserveNRes	Nachreservierung
SichBed	Sicherungsbedarf
Kapitalanlage_FV	DeckungsrückstellungFLVEnde
RisikoBeitrag	Risikobeitrag
SchlussAnteilFonds	SUAFonds
SolvabilitaetSoll	Solva
RfBEntnahmeNextYear	Bindung_RfB_lfdUeb
GebuchtBeitrag	Beitrag
AbschlussKosten	Übertrag in gekennzeichnete erw. LS-Größe möglich
BeitragRfB	RfB_Aus_Bonus
DirektGutschriften	Direktgutschrift
ErlebenLeistung	Aufw_Erleben nach Abzug von AblaufLeistung
RechZinsAnsammlung	Ansammlung_rmZins
SchlussUeberschuss	SUA
SchlussUeberschussGeb	Bindung_RfB_SUA
StornoLeistung	Aufw_RK
TodLeistung	Aufw_Tod
VerwaltungsKosten	Übertrag in gekennzeichnete erw. LS-Größe möglich
BestandEnde	Anzahl
Neugeschaeft	AnzahlNeu

Garantiezins	GarantiezinsBetrag
Wertveraenderung_FV	ZinsenFonds
LeistungAnsammlungErleben	VA_Erleben
LeistungAnsammlungTod	VA_Tod
LeistungAnsammlungStorno	VA_RK
SonstigeKosten	Übertrag in gekennzeichnete erw. LS-Größe möglich
Regulierungskosten2	Schadenregulierungskosten
Risikoergebnis	Risikoergebnis
Durchschnittsverzinsung	GewRechnungszins
DurchschnittsverzinsungRS	GewBilanzrechnungszins

3.2 Mustertransport-Funktion

```
var
  iRow: integer;
  b: Boolean;
  lv: ltab;

  DR, BonusDR, ZZR, Nachreservierung, FondsDR: real;
  ErlebenLeistung, AblaufLeistung: real;
begin
getltab('lv', lv);

iRow := getcuryear;

b := setvectattr(ltabgetrcn(lv, iRow, 'BestandEnde'), 'LifeResult',
'Anzahl', iRow);
b := setvectattr(ltabgetrcn(lv, iRow, 'Neugeschaeft'), 'LifeResult',
'AnzahlNeu', iRow);

DR := AnyToReal(ltabgetrcn(lv, iRow, 'BilanzReserve'));
BonusDR := AnyToReal(ltabgetrcn(lv, iRow, 'Bonusreserve'));
ZZR := AnyToReal(ltabgetrcn(lv, iRow, 'ZinsZusatzReserve'));
Nachreservierung := AnyToReal(ltabgetrcn(lv, iRow, 'BilanzReserveNRes'));
FondsDR := AnyToReal(ltabgetrcn(lv, iRow, 'Kapitalanlage_FV'));

ErlebenLeistung := AnyToReal(ltabgetrcn(lv, iRow, 'ErlebenLeistung'));
AblaufLeistung := AnyToReal(ltabgetrcn(lv, iRow, 'AblaufLeistung'));

DR := DR - FondsDR - BonusDR - ZZR - Nachreservierung;

{ Deckungsrückstellung }

b := setvectattr(NewAnySetRealValue(DR), 'LifeResult',
'DeckungsrückstellungEnde', iRow);
b := setvectattr(NewAnySetRealValue(DR + Nachreservierung), 'LifeResult',
'DeckungsrückstellungBilanzEnde', iRow);
b := setvectattr(NewAnySetRealValue(BonusDR), 'LifeResult',
'DeckungsrückstellungBonusEnde', iRow);
b := setvectattr(NewAnySetRealValue(ZZR), 'LifeResult',
'Zinszusatzreserve', iRow);
b := setvectattr(NewAnySetRealValue(Nachreservierung), 'LifeResult',
'Nachreservierung', iRow);
b := setvectattr(NewAnySetRealValue(FondsDR), 'LifeResult',
'DeckungsrückstellungFLVEnde', iRow);
b := setvectattr(ltabgetrcn(lv, iRow, 'SichBed'), 'LifeResult',
'Sicherungsbedarf', iRow);

b := setvectattr(ltabgetrcn(lv, iRow, 'Aktivierung'), 'LifeResult',
'AusstehendeAbschlusskosten', iRow);
```

```
b := setvectattr(ltabgetrcn(lv, iRow, 'Garantiezins'), 'LifeResult',
'GarantiezinsBetrag', iRow);

b := setvectattr(ltabgetrcn(lv, iRow, 'Wertveraenderung_FV'), 'LifeResult',
'ZinsenFonds', iRow);


{ RfB / SÜA }

b := setvectattr(ltabgetrcn(lv, iRow, 'RfBEntnahmeNextYear'), 'LifeResult',
'Bindung_RfB_lfdUeb', iRow);
b := setvectattr(ltabgetrcn(lv, iRow, 'SchlussUeberschussGeb'),
'LifeResult', 'Bindung_RfB_SUA', iRow);
b := setvectattr(ltabgetrcn(lv, iRow, 'SchlussAnteilFonds'), 'LifeResult',
'SUAFonds', iRow);
b := setvectattr(ltabgetrcn(lv, iRow, 'SchlussUeberschuss'), 'LifeResult',
'SUA', iRow);

{ Beiträge }

b := setvectattr(ltabgetrcn(lv, iRow, 'GebuchtBeitrag'), 'LifeResult',
'Beitrag', iRow);
b := setvectattr(ltabgetrcn(lv, iRow, 'BeitragsUebertrag'), 'LifeResult',
'Beitragsübertrag', iRow);
b := setvectattr(ltabgetrcn(lv, iRow, 'BeitragRfB'), 'LifeResult',
'RfB_Aus_Bonus', iRow);

b := setvectattr(ltabgetrcn(lv, iRow, 'RisikoBeitrag'), 'LifeResult',
'Risikobeitrag', iRow);

{ Aufwand für VF }

b := setvectattr(NewAnySetRealValue(AblaufLeistung), 'LifeResult',
'Aufw_Erleben', iRow);
b := setvectattr(NewAnySetRealValue(ErlebenLeistung - AblaufLeistung),
'LifeResult', 'Aufw_Rente', iRow);
b := setvectattr(ltabgetrcn(lv, iRow, 'TodLeistung'), 'LifeResult',
'Aufw_Tod', iRow);
b := setvectattr(ltabgetrcn(lv, iRow, 'StornoLeistung'), 'LifeResult',
'Aufw_RK', iRow);

b := setvectattr(ltabgetrcn(lv, iRow, 'LeistungAnsammlungErleben'),
'LifeResult', 'VA_Erleben', iRow);
b := setvectattr(ltabgetrcn(lv, iRow, 'LeistungAnsammlungTod'),
'LifeResult', 'VA_Tod', iRow);
b := setvectattr(ltabgetrcn(lv, iRow, 'LeistungAnsammlungStorno'),
'LifeResult', 'VA_RK', iRow);

{ Kosten }

b := setvectattr(ltabgetrcn(lv, iRow, 'AbschlussKosten'), 'CarryOver',
'Abschlusskosten', iRow);
```

```
b := setvectattr(ltabgetrcn(lv, iRow, 'VerwaltungsKosten'), 'CarryOver',
'Verwaltungskosten', iRow);
b := setvectattr(ltabgetrcn(lv, iRow, 'SonstigeKosten'), 'CarryOver',
'UnternehmenAlsGanzes', iRow);
b := setvectattr(ltabgetrcn(lv, iRow, 'Regulierungskosten2'), 'CarryOver',
'Schadenregulierungskosten', iRow);

b := setvectattr(ltabgetrcn(lv, iRow, 'AnsammlungsGuthaben'), 'LifeResult',
'AnsammlungsguthabenEnde', iRow);
b := setvectattr(ltabgetrcn(lv, iRow, 'RechZinsAnsammlung'), 'LifeResult',
'Ansammlung_rmZins', iRow);

b := setvectattr(ltabgetrcn(lv, iRow, 'DirektGutschriften'), 'LifeResult',
'Direktgutschrift', iRow);

b := setvectattr(ltabgetrcn(lv, iRow, 'Risikoergebnis'), 'LifeResult',
'Risikoergebnis', iRow);
b := setvectattr(ltabgetrcn(lv, iRow, 'SolvabilitaetSoll'), 'LifeResult',
'Solva', iRow);

b := setvectattr(ltabgetrcn(lv, iRow, 'Durchschnittsverzinsung'),
'LifeResult', 'GewRechnungszins', iRow);
b := setvectattr(ltabgetrcn(lv, iRow, 'DurchschnittsverzinsungRS'),
'LifeResult', 'GewBilanzrechnungszins', iRow);
end.
```